Lectures 10 & 11: Attention Layers, Transformers and Credibility Transformers

Deep Learning for Actuarial Modeling
Summer School 'Lifelong Learning'
Università Cattolica del Sacro Cuore, Milano

AUTHOR

Salvatore Scognamiglio, Marco Maggi, Mario V. Wüthrich, Ronald Richman

PUBLISHED

September 3, 2026

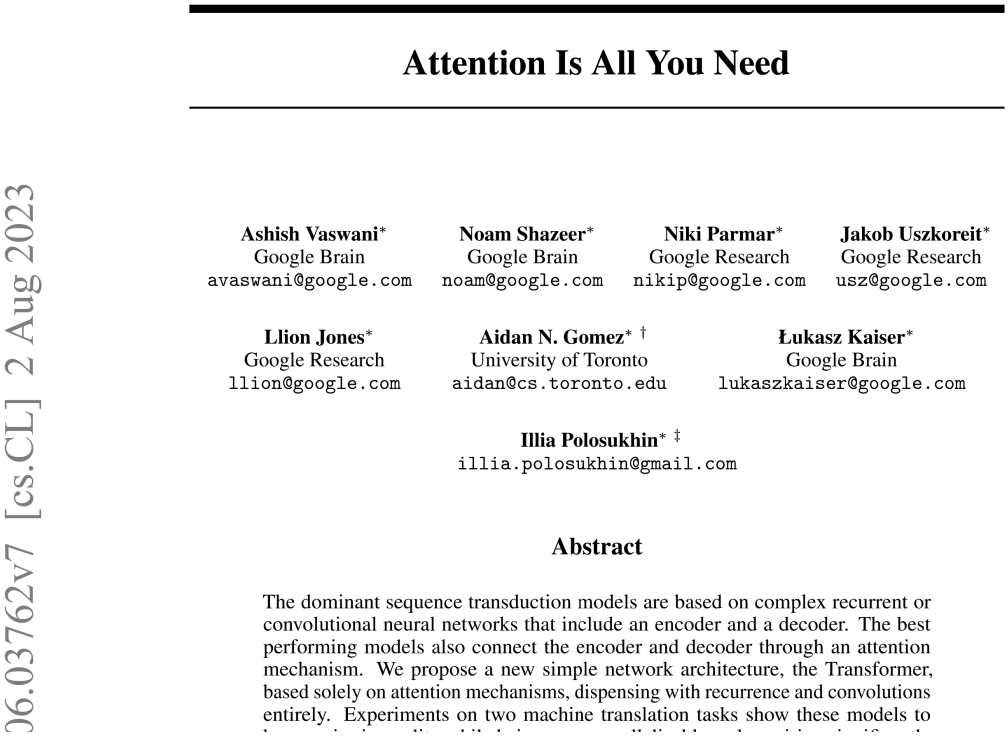
ABSTRACT

Attention layers are fundamental components of the Transformer architecture, which have achieved remarkable success across a wide range of tasks. This notebook introduces the core concepts of attention, it discusses positional encoding and CLS tokens, and it introduces the Transformer and the Credibility Transformer architectures.

1 Transformers - building blocks

OVERVIEW

- *Transformers* are a class of deep learning architectures that have revolutionized natural language processing (NLP).
- Transformers are at the core of the great success of large language models (LLMs).
- Transformer architectures replace traditional recurrent and convolutional structures by *attention layers*.
- Attention layers use *weighting schemes* to identify and prioritize the most relevant information and their interactions.
- The first section of this lecture introduces auxiliary building blocks before diving into the theory of Transformers.



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

06.03762v7 [cs.CL] 2 Aug 2023

Abstract

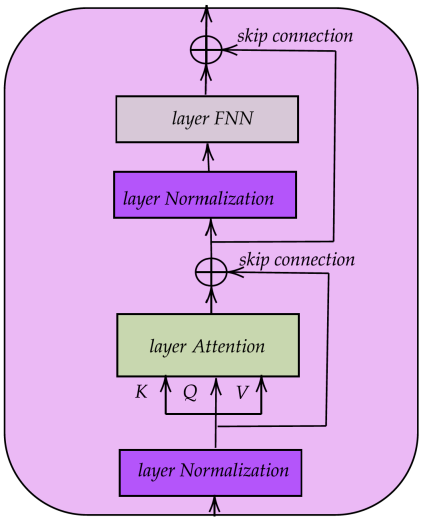
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to

- Transformers and attention layers were introduced in the celebrated work of Vaswani *et al.* (2017).

- People say that this is the most important deep learning paper of the last decade.

1.1 Basic components of Transformers

Transformers are sophisticated architectures that combine multiple layers. Before discussing the Transformer architecture, we introduce the key layers that serve as building blocks of these advanced architectures.



1.2 Time-distributed layer

OVERVIEW

- We generally work with *2D tensors* in this lecture given by *sequential data*

$$\boldsymbol{X}_{1:t} = \Big[\boldsymbol{X}_1, \ldots, \boldsymbol{X}_t\Big]^\top \in \mathbb{R}^{t \times q},$$

  with *time slices* $\boldsymbol{X}_u \in \mathbb{R}^q$ at time points $u = 1, \ldots, t$, and having q *channels*.
- A *time-distributed layer* applies the *identical* operation to every component (time slice) $\boldsymbol{X}_u$ of the sequential data $\boldsymbol{X}_{1:t} \in \mathbb{R}^{t \times q}$.
- We illustrate a *time-distributed FNN layer* in this section.

- Let $\boldsymbol{z}^{\text{FNN}} : \mathbb{R}^q \to \mathbb{R}^{q_1}$ be a FNN layer with q_1 units/neurons.
- A time-distributed FNN layer with q_1 units performs the mapping

$$\begin{aligned}\boldsymbol{z}^{\text{t-FNN}} &: \mathbb{R}^{t \times q} \to \mathbb{R}^{t \times q_1}, \\ \boldsymbol{X}_{1:t} &\mapsto \boldsymbol{z}^{\text{t-FNN}}(\boldsymbol{X}_{1:t}) = \big(\boldsymbol{z}^{\text{FNN}}(\boldsymbol{X}_1), \ldots, \boldsymbol{z}^{\text{FNN}}(\boldsymbol{X}_t)\big)^\top.\end{aligned}$$

- This transformation leaves the time dimension t unchanged.
- Important, the same parameters (network biases and weights) are shared across all time steps $1 \leq u \leq t$, i.e., the same transformation $\boldsymbol{z}^{\text{FNN}}$ is applied to all time slices $\boldsymbol{X}_u$.
- This makes the FNN layer a so-called *time-distributed* one.
- There is no interaction between different time slices $\boldsymbol{X}_u$ and $\boldsymbol{X}_s$, $u \neq s$!

1.3 Layer normalization

OVERVIEW

- *Layer normalization*, introduced by Ba, Kiros and Hinton (2016), is a technique used to improve the learning process, i.e., to robustify and accelerate convergence of SGD.
- A layer normalization is applied to the *individual* instances $\boldsymbol{X}_{i,1:t}$ across *all* covariate (feature) components – this is not affected by the batch size.
- In contrast, *batch normalization* of Ioffe and Szegedy (2015) is applied for a fixed covariate component across all instances in the batch; additionally, batch normalization often involves a moving average mechanism for having stability across multiple batches.

- For sequential data $\boldsymbol{X}_{1:t}$, a *layer normalization* is a mapping

$$\begin{aligned} \boldsymbol{z}^{\text{norm}} &: \mathbb{R}^{t\times q} \to \mathbb{R}^{t\times q}, \\ \boldsymbol{X}_{1:t} &\mapsto \boldsymbol{z}^{\text{norm}}(\boldsymbol{X}_{1:t}) = \left(\gamma_j \left(\frac{X_{u,j} - \bar{X}_u}{\sqrt{s_u^2 + \epsilon}}\right) + \delta_j\right)_{1\le u\le t, 1\le j\le q}, \end{aligned}$$

  where $\epsilon > 0$ is a small constant added for numerical stability.
- The empirical means $\bar{X}_u \in \mathbb{R}$ and variances $s_u^2 \in \mathbb{R}_+$ are computed within the time slices

$$\bar{X}_u = \frac{1}{q}\sum_{j=1}^{q} X_{u,j} \qquad \text{and} \qquad s_u^2 = \frac{1}{q}\sum_{j=1}^{q}(X_{u,j} - \bar{X}_u)^2.$$

- $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_q)^\top \in \mathbb{R}^q$ and $\boldsymbol{\delta} = (\delta_1, \ldots, \delta_q)^\top \in \mathbb{R}^q$ are two vectors of trainable parameters.
- This adds $2q$ trainable parameters.

1.4 Example: layer normalization & time-distributed FNN

The following code shows a layer normalization followed by a time-distributed FNN layer processing the input tensor $\boldsymbol{X}_{1:t} \in \mathbb{R}^{t \times q}$.

The following `TD_1layer` function takes the following arguments:

- `seed`: to ensure reproducibility of results (initialization of weights);
- `input_size`: input dimension of the 2D tensor (sequence length t and number of channels q);
- `td_units`: number of neurons in the `time_distributed` FNN layer.

The layer normalization layer is called `layer_layer_normalization`; see next code.

```
TD_1layer <- function(seed, input_size, td_units) {
    k_clear_session(); set.seed(seed); set_random_seed(seed)

    # input layer
    Design <- layer_input(shape = input_size, dtype = 'float32')

    #  layer normalization and time-distributed FNN layer
    Output <- Design %>%
       layer_layer_normalization() %>%
       time_distributed(layer_dense(units = td_units, activation =
         'relu'))

    # define model
    keras_model(inputs = Design, outputs = Output)
}
```

- To understand its functioning and parametrization, it is useful to have a numerical example.

- This example uses an input tensor of dimension $(t, q) = (10, 5)$ and the time-distributed FNN layer has $q_1 = 8$.

```
model = TD_1layer(
  seed = 100,
  input_size = c(10, 5),    # time steps t=10 and channels q=5
  td_units = 8
  )
```

- The time-distributed FNN layer has $(q+1)q_1 = 6 \cdot 8 = 48$ parameters.
- The layer normalization adds another $2q = 10$ (time-distributed) parameters to the architecture.
- The sequence length t does not impact the number of parameters because all operations are performed in a time-distributed manner.

```
summary(model)
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 10, 5)]           0
 layer_normalization (LayerNormali  (None, 10, 5)      10
 zation)
 time_distributed (TimeDistributed  (None, 10, 8)      48
 )
=================================================================
Total params: 58 (232.00 Byte)
Trainable params: 58 (232.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

- Up to this stage, the time slices $\boldsymbol{X}_u$ have not interacted, yet!

- In fact, this can be seen as a simultaneous pre-processing of all time slices before they start to interact.

1.5 Attention layer

OVERVIEW

- *Attention layers* are the *key building blocks* of *Transformers*.
- Attention layers are designed to identify the most relevant information in the input data.
- The central idea is to learn a weighting scheme that prioritizes the most important parts and their interactions.
- Different attention mechanisms are available in the literature. Our focus is on the most commonly used variant called *scaled dot-product attention*.

1.6 Sequential input data

Attention layers were first designed for sequential input data.

The starting point is an input sequence of elements:

$$\boldsymbol{X}_{1:t} = \left[\boldsymbol{X}_1, \ \ldots, \ \boldsymbol{X}_t\right]^\top \in \mathbb{R}^{t \times q},$$

where (again):

- t is length of the sequence,
- q is dimension of the series (number of channels),
- $\boldsymbol{X}_u \in \mathbb{R}^q$ is the u-th time slice, with $1 \leq u \leq t$.

Below, we are also going to apply attention to tabular input data, but this will require suitable pre-processing.

1.7 Query, key and value

In order to be applied, an attention layer requires the following elements:

- Queries $\boldsymbol{q}_u \in \mathbb{R}^q,\ 1 \leq u \leq t$:
  - The query $\boldsymbol{q}_u$ represents what the u-th time slice is trying to find in the overall input.
  - It acts like *question asking*: "What information is relevant to me?"
- Keys $\boldsymbol{k}_u \in \mathbb{R}^q,\ 1 \leq u \leq t$:
  - The key $\boldsymbol{k}_u$ describes what the u-th time slice can offer to others.
  - It functions like *label indicating*: "This is the kind of information I carry".
- Values $\boldsymbol{v}_u \in \mathbb{R}^q,\ 1 \leq u \leq t$:
  - The value $\boldsymbol{v}_u$ contains the actual content that can be shared.
  - When a query matches a key, the corresponding value is passed along as useful context.

- Colloquially speaking, the input data $\boldsymbol{X}_{1:t}$ is going to be equipped with *queries* $\boldsymbol{q}_{1:t}$ and *keys* $\boldsymbol{k}_{1:t}$ that communicate with each other.

> **ATTENTION MECHANISM**
>
> The *query* $\boldsymbol{q}_u$ searches for a *key* $\boldsymbol{k}_s$ that gives a match. E.g., in a sentence, the query of the subject 'car' tries to find a verb 'accelerate' in the sentence. This would give a match for a dangerous driving maneuver, and a high attention is paid to the corresponding *value* $\boldsymbol{v}_s$.

- Queries, keys and values are represented by the three 2D tensors

$$
\begin{aligned}
Q := \boldsymbol{q}_{1:t} &= [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_t]^\top \in \mathbb{R}^{t \times q}, \\
K := \boldsymbol{k}_{1:t} &= [\boldsymbol{k}_1, \ldots, \boldsymbol{k}_t]^\top \in \mathbb{R}^{t \times q}, \\
V := \boldsymbol{v}_{1:t} &= [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_t]^\top \in \mathbb{R}^{t \times q}.
\end{aligned}
$$

▷ Their explicit construction will be discussed later, below.

- Careful: q is the number of channels and $\boldsymbol{q}_u$ are the queries.

1.8 Attention head

- The *attention head* H is defined by the mapping

$$(Q, K, V) \;\mapsto\; H = H(Q, K, V) \;\in\; \mathbb{R}^{t\times q},$$

with scaled *dot-product attention*

$$H = A\,V = \operatorname{softmax}\left(\frac{QK^\top}{\sqrt{q}}\right)V \;\in\; \mathbb{R}^{t\times q}.$$

- The *attention matrix* $A \in \mathbb{R}^{t\times t}$ is obtained by applying the softmax function *row-wise* to the matrix $A^\circ := QK^\top/\sqrt{q}$, that is,

$$A = \operatorname{softmax}(A^\circ), \qquad \text{with} \quad a_{u,s} = \frac{\exp(a^\circ_{u,s})}{\sum_{k=1}^{t}\exp(a^\circ_{u,k})} \in (0,1).$$

$\Longrightarrow$ This ensures that the rows sums of A are equal to one.

1.9 Discussion of attention heads

- Recalling the notation. We have query and key tensors

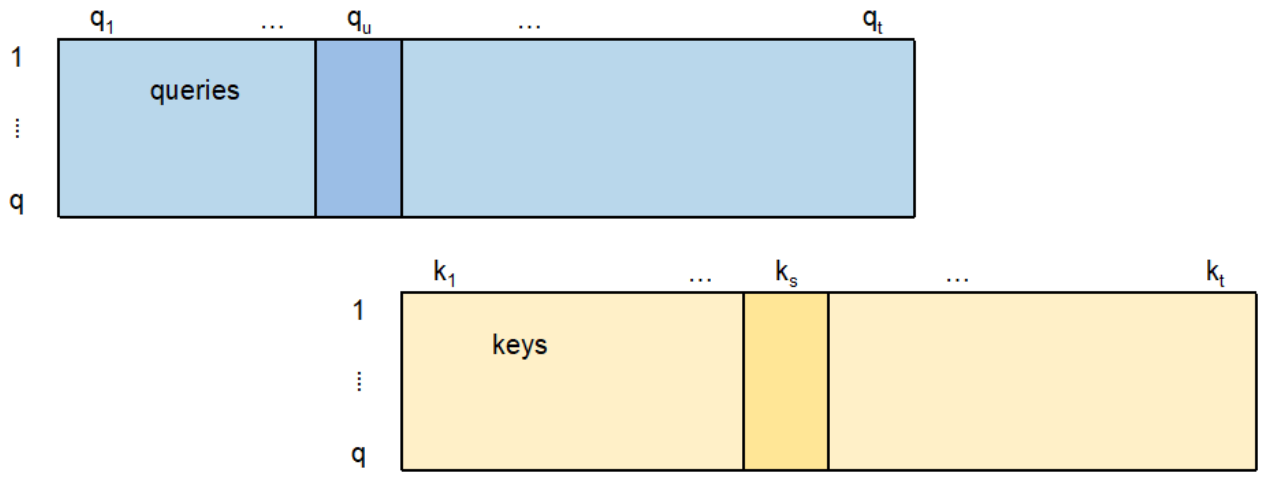
$$Q = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_t]^\top \in \mathbb{R}^{t\times q} \quad \text{and} \quad K = [\boldsymbol{k}_1, \ldots, \boldsymbol{k}_t]^\top \in \mathbb{R}^{t\times q}.$$

The elements $a^\circ_{u,s}$ of matrix A° are given by the dot-product

$$a^\circ_{u,s} = \frac{1}{\sqrt{q}}\,\boldsymbol{q}_u^\top \boldsymbol{k}_s = \frac{1}{\sqrt{q}}\,\langle \boldsymbol{q}_u, \boldsymbol{k}_s\rangle = \boldsymbol{q}_u \cdot \boldsymbol{k}_s/\sqrt{q};$$

these are three ways to write a scalar product; see also next figure.

- The scaling factor $\sqrt{q}$ removes the input dimension dependence. I.e., it prevents the dot-product operation from being too flat or too spiky.
- Each entry of the attention head $H = A\,V$ is a weighted average of the columns of the value matrix V. The weights $a_{u,s}$ determine the *importance* of each row vector $\boldsymbol{v}_s$ of V.



- If the query $\boldsymbol{q}_u$ points into the same direction as the key $\boldsymbol{k}_s$, we receive a large attention weight $a_{u,s}$ (supposed all vectors have roughly the same length).
- This implies that the corresponding entry $\boldsymbol{v}_s$ on the s-th row of the value matrix V receives a big attention, i.e., the information $\boldsymbol{v}_s$ at time s is important for period u (the query $\boldsymbol{q}_u$ at time u).

1.10 Multi-head attention

OVERVIEW

- An attention head H can be interpreted like a single neuron in a FNN layer because it uses a single set of parameters.
- Similar to FNN layers, we can have *multiple attention heads* H_j *in parallel* that perform the attention mechanism with different sets of parameters.
- This give the *multi-head attention mechanism*, allowing the model to focus more effectively on different parts of the input sequence simultaneously.

- The *multi-head attention* mechanism applies $n_h \geq 2$ attention heads to the sequential input $\boldsymbol{X}_{1:t}$ in parallel.
- Assume the j-th attention head is given by the query, key and value Q_j, K_j, and V_j, respectively, defining the j-th attention head

$$H_j = H_j(\boldsymbol{X}_{1:t}) = A_j V_j = \text{softmax}\left(Q_j K_j^\top / \sqrt{q}\right) V_j \in \mathbb{R}^{t \times q}.$$

- These heads are concatenated and linearly transformed

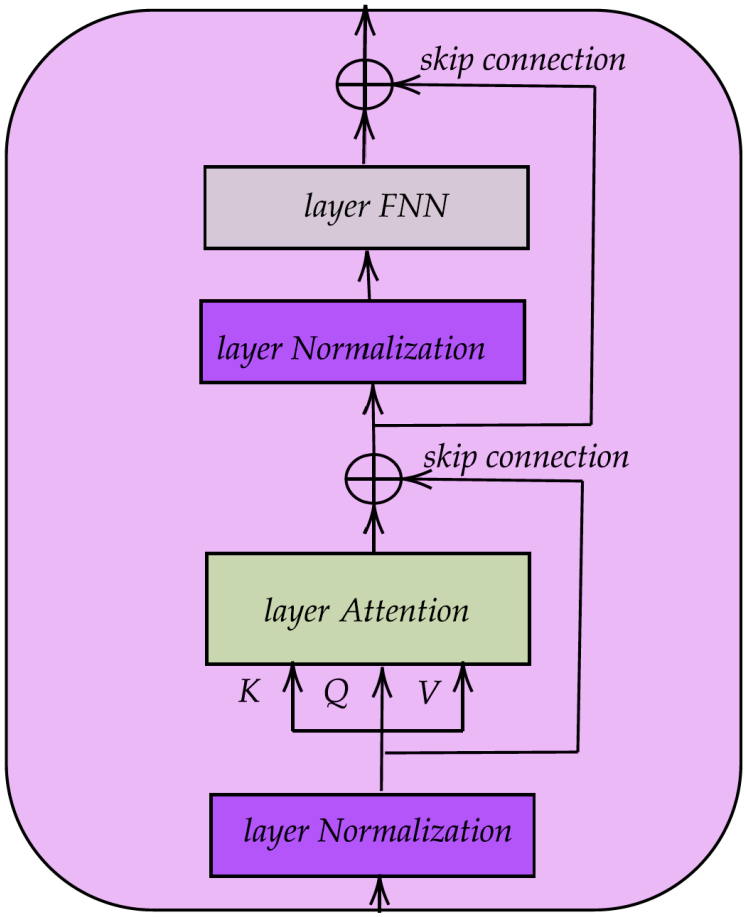
$$H_{\text{MH}}(\boldsymbol{X}_{1:t}) = \text{Concat}\left(H_1, H_2, \ldots, H_{n_h}\right) W \in \mathbb{R}^{t \times q},$$

  with an output weight matrix $W \in \mathbb{R}^{n_h q \times q}$.
- This concatenation and linear transformation is performed to ensure that the multi-head attention $H_{\text{MH}} \in \mathbb{R}^{t \times q}$ has the same dimension (t, q) as the input.
- This multi-head attention is further processed as described below.

2 Transformers for sequential data

OVERVIEW

- We are now ready to dive into the theory of Transformers for sequential data.
- This applies the attention mechanism precisely using the interpretation as discussed above.
- We present the so-called *self-attention mechanism* by letting the query, key and value be functions of the input tensor $\boldsymbol{X}_{1:t}$.
- This self-attention mechanism uses time-distributed FNNs to construct the queries, keys and values from the sequential input data.



2.1 Transformer layer

- Consider input data with a sequential structure represented in a 2D tensor form $\boldsymbol{X}_{1:t} \in \mathbb{R}^{t \times q}$.
- The 1st module of the Transformer architecture is a *layer normalization*.
- The 2nd module of the Transformer architecture is the *attention layer*.
- To apply the attention mechanism to the (layer normalized) input data $\boldsymbol{X}_{1:t}$, we need to construct the query Q, key K, and value V matrices.
- We select *3 time-distributed* q-dimensional FNNs

$$\boldsymbol{z}_{\kappa}^{\text{t-FNN}} : \mathbb{R}^{t \times q} \to \mathbb{R}^{t \times q}, \qquad \boldsymbol{X}_{1:t} \mapsto \boldsymbol{z}_{\kappa}^{\text{t-FNN}}(\boldsymbol{X}_{1:t}),$$

for the query, key and value $\kappa = Q, K, V$, respectively.

- These give us the time-slices for fixed time points $1 \leq u \leq t$

$$\begin{aligned}
\boldsymbol{q}_u &= \boldsymbol{z}_Q^{\text{FNN}}(\boldsymbol{X}_u) = \phi_Q\left(\boldsymbol{w}_0^{(Q)} + W^{(Q)}\boldsymbol{X}_u\right) \in \mathbb{R}^q, \\
\boldsymbol{k}_u &= \boldsymbol{z}_K^{\text{FNN}}(\boldsymbol{X}_u) = \phi_K\left(\boldsymbol{w}_0^{(K)} + W^{(K)}\boldsymbol{X}_u\right) \in \mathbb{R}^q, \\
\boldsymbol{v}_u &= \boldsymbol{z}_V^{\text{FNN}}(\boldsymbol{X}_u) = \phi_V\left(\boldsymbol{w}_0^{(V)} + W^{(V)}\boldsymbol{X}_u\right) \in \mathbb{R}^q,
\end{aligned}$$

  with corresponding network weights, biases, and activation functions.
- This reads in 2D tensor notation as

$$\begin{aligned}
Q &= \boldsymbol{z}_Q^{\text{t-FNN}}(\boldsymbol{X}_{1:t}) = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_t]^\top \in \mathbb{R}^{t\times q}, \\
K &= \boldsymbol{z}_K^{\text{t-FNN}}(\boldsymbol{X}_{1:t}) = [\boldsymbol{k}_1, \ldots, \boldsymbol{k}_t]^\top \in \mathbb{R}^{t\times q}, \\
V &= \boldsymbol{z}_V^{\text{t-FNN}}(\boldsymbol{X}_{1:t}) = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_t]^\top \in \mathbb{R}^{t\times q}.
\end{aligned}$$

- This is solely based on the time-series $\boldsymbol{X}_{1:t}$ (*self-attention*) and every time slice uses the same parameters (time-distributed FNNs).

  $\Longrightarrow$ There is no interaction across time slices, yet!

- The *attention head* received from input $\boldsymbol{X}_{1:t}$ is given by

$$H = H(\mathbf{X}_{1:t}) = \text{softmax}\left(QK^\top/\sqrt{q}\right)V \in \mathbb{R}^{t\times q}.$$

- Next, we add a so-called *skip connection*

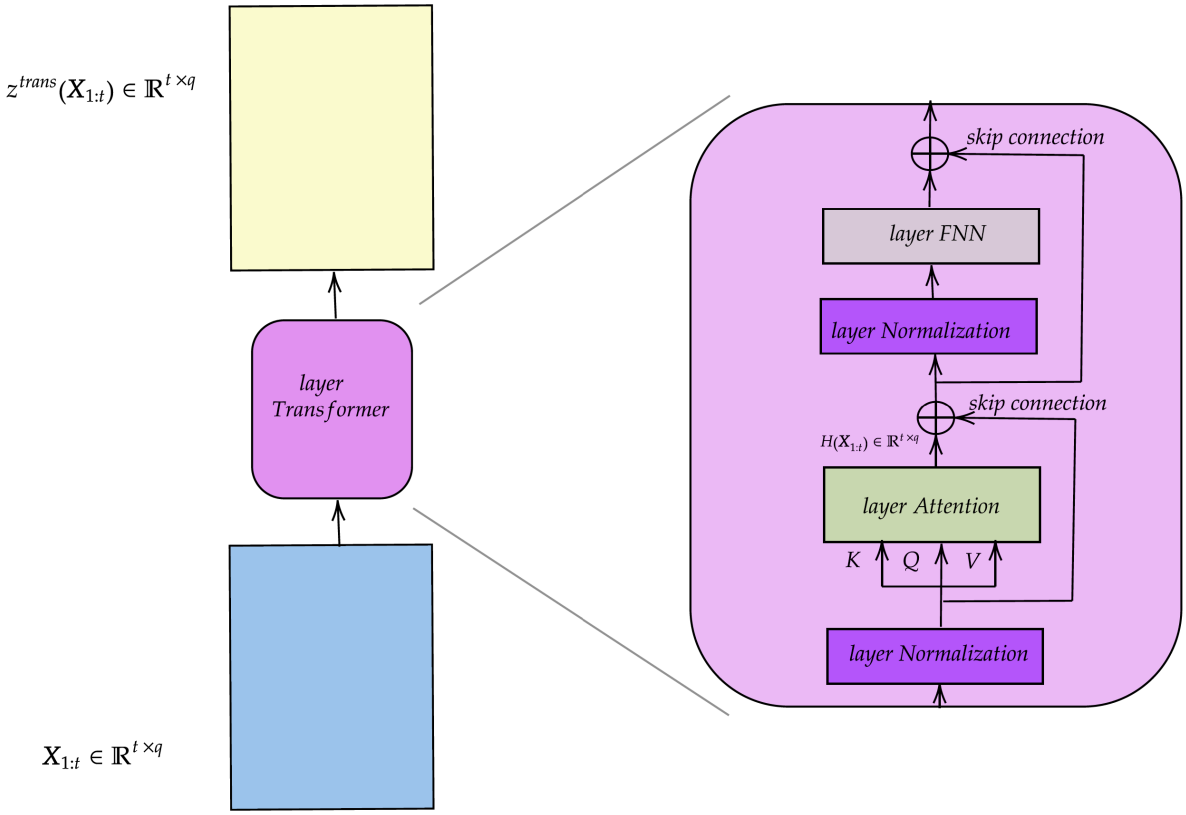
$$\boldsymbol{z}^{\text{skip}}(\boldsymbol{X}_{1:t}) := \boldsymbol{X}_{1:t} + H(\boldsymbol{X}_{1:t}) \in \mathbb{R}^{t\times q}.$$

  It's like a higher order perturbation in a Taylor series.
- Finally, we layer-normalize, pass through a time-distributed FNN layer and skip-connect again. This defines the *Transformer layer*

$$\boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:t}) = \boldsymbol{z}^{\text{skip}}(\boldsymbol{X}_{1:t}) + \left(\boldsymbol{z}^{\text{t-FNN}} \circ \boldsymbol{z}^{\text{norm}}\right)\left(\boldsymbol{z}^{\text{skip}}(\boldsymbol{X}_{1:t})\right) \in \mathbb{R}^{t\times q}.$$

- The Transformer layer preserves the dimension of the 2D input tensor, but it *transforms* its elements by the attention mechanism.

2.2 Illustration of a Transformer layer



2.3 Remarks on Transformer layers

- The Transformer layer encodes (transforms) the sequential data $\boldsymbol{X}_{1:t} \in \mathbb{R}^{t \times q}$ into a new 2D tensor $\boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:t}) \in \mathbb{R}^{t \times q}$ of the same dimension.
- One can easily replace the (single) attention head H by a multi-head attention H_{MH}.
- Multiple Transformer layers can be stacked (composed). This leads to deep Transformer architectures.
- There are two critical issues:

  1. The Transformer layer does not have any notion of time.
  2. The Transformer layer does not respect time-causality.

2.4 Positional index

- To deal with the first issue of notion of time: add *positional indices* to the time-series data $\boldsymbol{X}_{1:t}$.
- A simple way to do so is to extend the time slices to

$$\boldsymbol{X}_u = (X_{u,1}, \ldots, X_{u,q}, u/t)^\top \in \mathbb{R}^{q+1},$$

  thus, we add an extra channel for the normalized positional encoding.
- Following Vaswani *et al.* (2017), it can be beneficial to add multiple positional encodings in different units. This facilitates learning the right functional forms. E.g., add sine and cosine functions

$$\boldsymbol{X}_u = (X_{u,1}, \ldots, X_{u,q}, u/t, \sin(2\pi t/u), \cos(2\pi t/u))^\top \in \mathbb{R}^{q+3},$$

- The latter can be done for different frequencies, also known as *Rotary Position Embedding* (RoPE).

2.5 Time-causality

- The raw elements $a^\circ_{u,s}$ entering the attention matrix A are given by the dot-product operation

$$a^\circ_{u,s} = \boldsymbol{q}_u \cdot \boldsymbol{k}_s / \sqrt{q}.$$

- The attention matrix $A \in \mathbb{R}^{t\times t}$ is obtained by applying the softmax function *row-wise* to the matrix $A^\circ := QK^\top/\sqrt{q}$, that is,

$$A = \mathrm{softmax}(A^\circ), \qquad \text{with} \quad a_{u,s} = \frac{\exp(a^\circ_{u,s})}{\sum_{k=1}^t \exp(a^\circ_{u,k})} \in (0,1).$$

- This allows for interactions between any query $\boldsymbol{q}_u$ and any key $\boldsymbol{k}_s$.
- In time-causal situations, the query $\boldsymbol{q}_u$ should not be allowed to access the key $\boldsymbol{k}_s$ if $s > u$ (information after time u).

- A time-causal Transformer layer is received by *masking* the attention weights that violate time-causality.
- Therefore, set

$$a^{\circ}_{u,s} = \begin{cases} \boldsymbol{q}_u \cdot \boldsymbol{k}_s / \sqrt{q} & \text{if } s \leq u, \\ -\infty & \text{if } s > u. \end{cases}$$

- This implies

$$a_{u,s} = \exp(-\infty) = 0 \qquad \text{for all } s > u.$$

- Therefore, this time-causal version disregards all information $\boldsymbol{X}_s$ and $\boldsymbol{v}_s$ that occurred after time u (query $\boldsymbol{q}_u$).
- Since the attention layer is the only place where time slices interact in the Transformer layer, this does the job for time-causality.

2.6 Example of a Transformer layer

The following code defines a (non-time causal) Transformer layer.

The `transf_layer` function takes the following arguments:

- `seed`: to ensure reproducibility of results;
- `input_size`: the input dimension of the sequential data (sequence length t and number of channels q).

The attention layer in `keras` needs special care because the order of the inputs to `layer_attention` is (`query`, `value`, `key`).

```
transf_layer <- function(seed, input_size) {
  k_clear_session(); set.seed(seed); set_random_seed(seed)
  q = input_size[2]
  Design = layer_input(shape = c(input_size[1], q), dtype = 'float32')

  # Deriving Q, K, V (we drop the initial layer normalization)
  Q = Design %>% time_distributed(layer_dense(units = q, activation =
        "tanh"))
  K = Design %>% time_distributed(layer_dense(units = q, activation =
        "tanh"))
  V = Design %>% time_distributed(layer_dense(units = q, activation =
        "tanh"))

  # Attention mechanism
  Attention <- list(Q, V, K) %>% layer_attention(use_scale = FALSE)
  #
  skip_1 <- layer_add(list(Attention, Design))
  skip_2 <- skip_1 %>% layer_layer_normalization() %>%
        time_distributed(layer_dense(units = q))
  #
  Output <- layer_add(list(skip_1, skip_2))
  keras_model(inputs = Design, outputs = Output)
  }
```

- The next example showcases how to create a single-headed and single-layered Transformer architecture.

```
#
model = transf_layer(
  seed = 100,
  input_size = c(10, 5)    # length t=10 of time-series and q=5 channels
  )
```

- This architecture has 130 parameters (this is independent of t!):
  - 90 to create the query, key and value;
  - 10 for the layer normalization;
  - 30 for the time-distributed FNN.
- `use_scale = TRUE` gives a trainable scaling in the attention scores.

- The output of this architecture is a 2D tensor of the same dimension $(t, q) = (10, 5)$ as the input.

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 10, 5)]           0
 time_distributed (TimeDistributed)  (None, 10, 5)     30
 time_distributed_2 (TimeDistributed)  (None, 10, 5)   30
 time_distributed_1 (TimeDistributed)  (None, 10, 5)   30
 attention (Attention)       (None, 10, 5)             0
['time_distributed[0][0]',
'time_distributed_2[0][0]',
'time_distributed_1[0][0]']
 add (Add)                   (None, 10, 5)             0
['attention[0][0]',
'input_1[0][0]']
 layer_normalization (LayerNormalization)  (None, 10, 5)  10
['add[0][0]']
 time_distributed_3 (TimeDistributed)  (None, 10, 5)   30
['layer_normalization[0][0]']
 add_1 (Add)                 (None, 10, 5)             0
['add[0][0]',
'time_distributed_3[0][0]']
=================================================================
Total params: 130 (520.00 Byte)
Trainable params: 130 (520.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

3 Transformers for tabular data

OVERVIEW

- The excellent predictive performance of Transformers on sequential data motivated modelers to also apply Transformer architectures to tabular input data $\boldsymbol{X}$.
- There are several modifications of Transformers to tabular inputs. We focus on the approach proposed by Gorishniy, Kotelnikov and Babenko (2024).
- The key idea lies in *feature tokenization* which converts all original covariates – both categorical and continuous ones – into the same format resulting in a 2D tensor.

3.1 Input tensor construction

- Consider the *raw* tabular (vector) inputs $\boldsymbol{X} = (X_1, \ldots, X_q)^\top$.
- Assume this raw input data consists of:
  - q_c categorical covariates $(X_j)_{j=1}^{q_c}$, and
  - $q - q_c$ continuous (numerical) covariates $(X_j)_{j=q_c+1}^{q}$.
- We *embed* both types of covariates into a b-dimensional Euclidean space $\mathbb{R}^b$, where b is a hyper-parameter selected by the modeler.
- For categorical variables, this has already been presented in the chapter of *entity embedding*, and it is frequently used in FNN architectures, but also in large language models (LLMs).
- For continuous variables, we present an interesting method that is very effective in many problems.

3.2 Categorical covariates: entity embedding, revisited

- For categorical covariates we use entity embedding.
- Entity embedding was introduced in the lectures on Wednesday.
- Assume that the categorical covariate X_j takes the levels in the finite set $\mathcal{A}_j = \{a_1, \ldots, a_{K_j}\}$.
- An entity embedding of X_j is given by the mapping

$$\boldsymbol{e}_j^{\text{EE}} : \mathcal{A}_j \to \mathbb{R}^b, \qquad X_j \mapsto \boldsymbol{e}_j^{\text{EE}} := \boldsymbol{e}_j^{\text{EE}}(X_j).$$

- All categorical covariates share the same embedding dimension b.
- The categorical covariates involve $b \sum_{j=1}^{q_c} K_j$ embedding weights to be learned.

3.3 Continuous covariates and 2D tensors

- For the continuous covariates X_j, $j = q_c + 1, \ldots, q$, we perform a b-dimensional *embedding via FNNs*

$$\boldsymbol{z}_j^{\text{FNN}} : \mathbb{R} \to \mathbb{R}^b, \qquad X_j \mapsto \boldsymbol{z}_j := \boldsymbol{z}_j^{\text{FNN}}(X_j),$$

  e.g., $\boldsymbol{z}_j^{\text{FNN}}(\cdot)$ can be FNNs of depth 2 for covariate X_j.
- Similarly to generalized additive models (GAMs), this facilitates marginal modeling of covariates X_j for non-linear structures.
- I.e., a main difficulty in fitting plain-vanilla FNNs with SGD is to find suitable functional forms for the input components. This b-dimensional embedding is a sort of pre-processing to bring all continuous components into a suitable functional form.
- Another successful proposal by Gorishniy, Rubachev and Babenko (2022) is called *piecewise linear encoding* (PLE). In some sense, it can be understood as a continuous version of categorical binning.

- Collecting all embedded categorical and continuous covariates gives us the 2D tensor

$$\boldsymbol{X}^{\circ}_{1:q} := \left[\boldsymbol{e}^{\text{EE}}_1, \ldots, \boldsymbol{e}^{\text{EE}}_{q_c}, \boldsymbol{z}_{q_c+1}, \ldots, \boldsymbol{z}_q\right]^\top \in \mathbb{R}^{q \times b}.$$

- This 2D tensor has the correct shape to enter a Transformer:
  - q plays the role of the time dimension, and
  - b is the number of channels.
- Naturally, there is no time-causal structure here, i.e., covariate components can be arbitrarily permuted before entering the Transformer layer.

3.4 CLS token

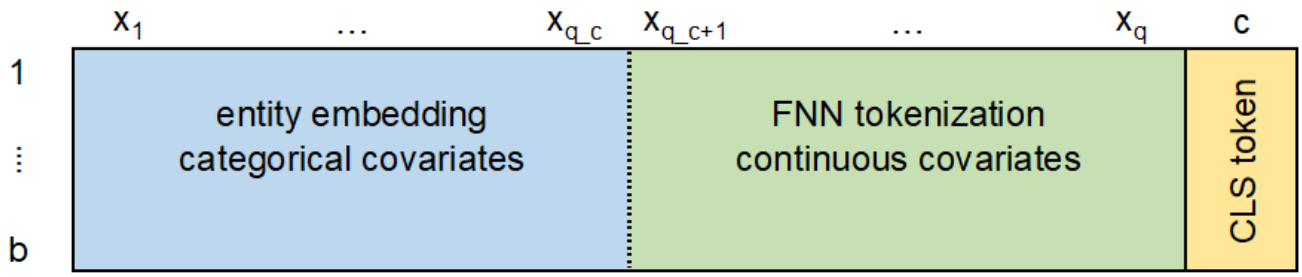
- Before processing the 2D tensor $\boldsymbol{X}^{\circ}_{1:q}$ of the embedded covariates through a Transformer layer, we add a special token to the input tensor.
- This special token is going to be used to *encode* the input data $\boldsymbol{X}$.
- This idea is borrowed from Devlin *et al.* (2019) in BERT (bidirectional encoder representations from transformers), which adds a special token – called *classify (CLS) token* – to the tensor to encode the probabilities of the next word in a text.
- Our CLS token will not represent a probability but a real number. Nevertheless, we keep the notion *CLS token* to emphasize its similar role as in BERT.
- The CLS token $\boldsymbol{c} = (c_1, \ldots, c_b)^\top \in \mathbb{R}^b$ is added to every column of the input tensor $\boldsymbol{X}^{\circ}_{1:q} \in \mathbb{R}^{q \times b}$.

3.5 The augmented input tensor

- The *augmented input tensor* is defined by

$$\boldsymbol{X}_{1:q+1} = \begin{bmatrix} \boldsymbol{X}^{\circ}_{1:q} \\ \boldsymbol{c}^\top \end{bmatrix} = \left[\boldsymbol{e}^{\text{EE}}_1, \ldots, \boldsymbol{e}^{\text{EE}}_{q_c}, \boldsymbol{z}_{q_c+1}, \ldots, \boldsymbol{z}_q, \boldsymbol{c}\right]^\top \in \mathbb{R}^{(q+1) \times b}.$$

- Scalar c_k is used to encode the k-th column of the input tensor $\boldsymbol{X}^{\circ}_{1:q}$.



- $q+1$ plays the role of the time dimension, and b is the number of channels; there is no time-causality here.

3.6 The CLS token encoding

- The CLS token $\boldsymbol{c}$ is randomly initialized not carrying any information *before* interacting with the other covariate components.
- We process the augmented input tensor through a Transformer layer

$$\boldsymbol{z}^{\text{trans}} : \mathbb{R}^{(q+1)\times b} \to \mathbb{R}^{(q+1)\times b}, \qquad \boldsymbol{X}_{1:q+1} \mapsto \boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:q+1}).$$

- The last row of the Transformer output $\boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:q+1})$ corresponds to the *transformed CLS token*, and we set

$$\boldsymbol{c}^{\text{trans}}(\boldsymbol{X}) := \boldsymbol{z}^{\text{trans}}_{q+1}(\boldsymbol{X}_{1:q+1}) \in \mathbb{R}^b.$$

TRANSFORMED CLS TOKEN

After interacting with the other covariate components through the attention mechanism, this transformed CLS token $\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})$ has *learned an encoded representation* of the original covariates $\boldsymbol{X}$.

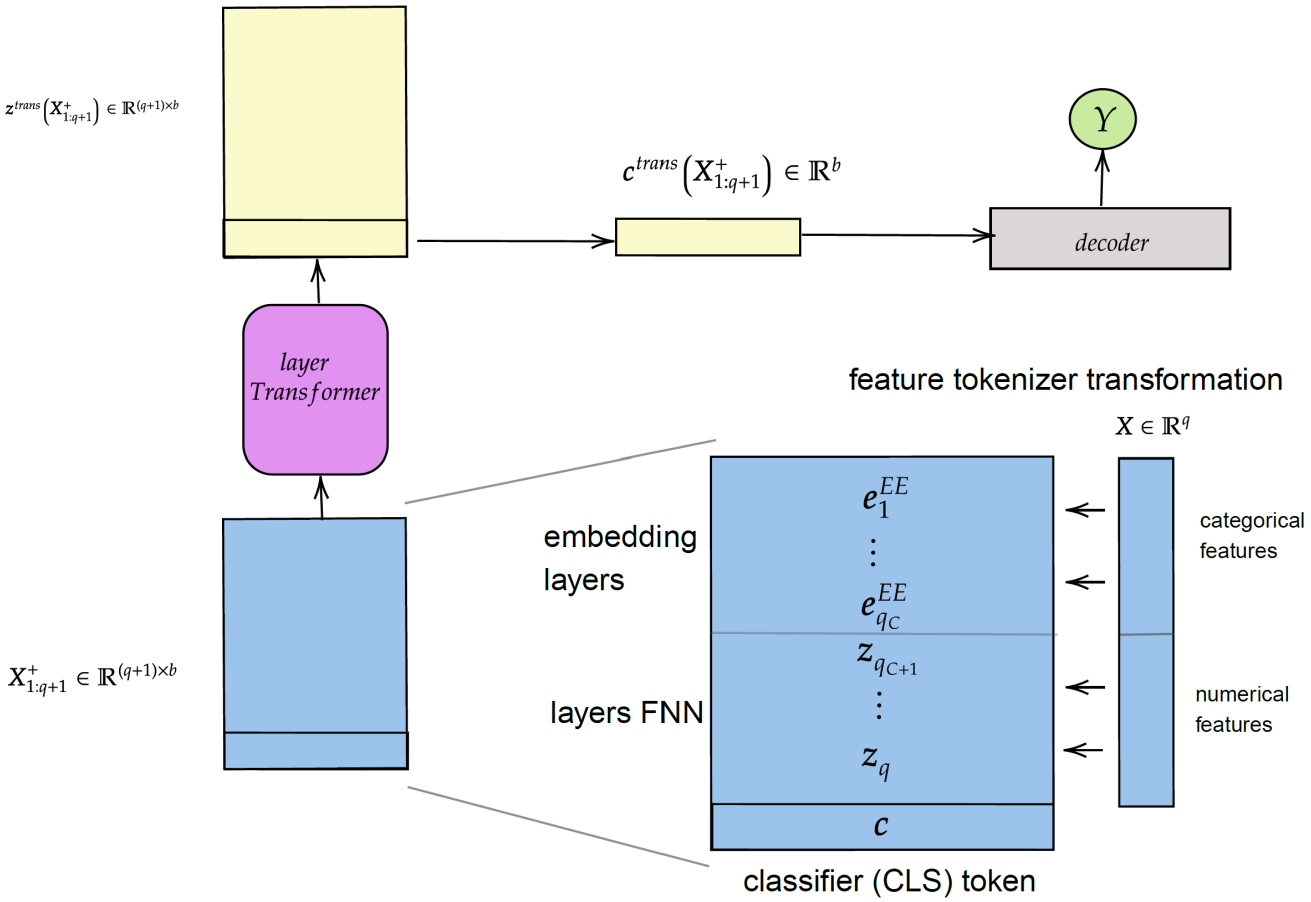
- For further processing, only this encoded representation is forwarded

$$\boldsymbol{X} \mapsto \boldsymbol{c}^{\text{trans}}(\boldsymbol{X}) \mapsto \cdots.$$

- The final step is to decode the transformed CLS token $\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})$ to a prediction for the response.
- For example, for insurance claims modeling, one often selects the log-link, and one may set

$$\mu^{\text{trans}}(\boldsymbol{X}) = \exp\left(\left(\boldsymbol{z}^{\text{FNN}} \circ \tanh \circ \boldsymbol{z}^{\text{norm}}\right)\left(\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})\right)\right),$$

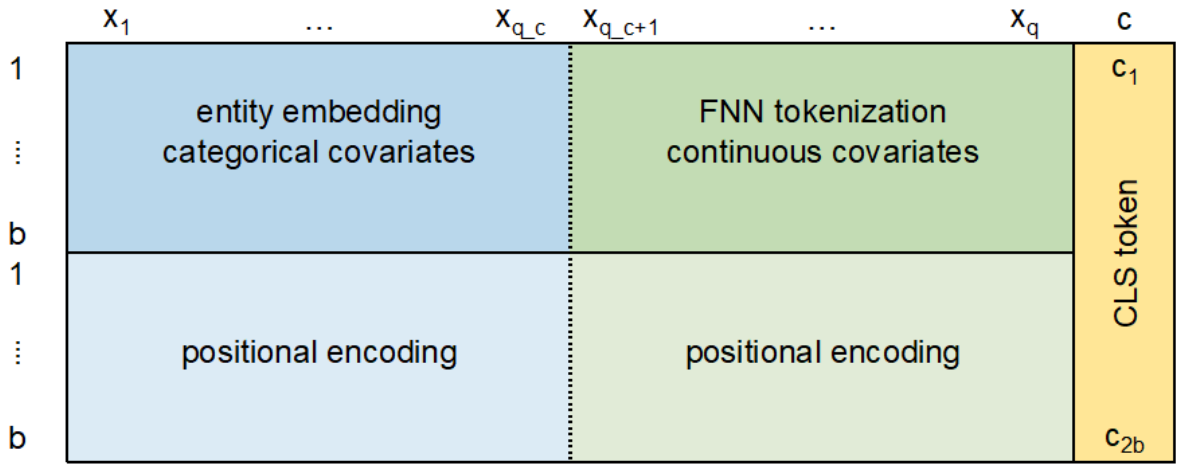
  which includes a layer normalization, a hyperbolic tangent activation, and a FNN layer with a one-dimensional output.
- This architecture is illustrated in the next graph.



3.7 Remark positional encodings

- For the tabular input data $\boldsymbol{X} = (X_1, \ldots, X_q)^\top$, position $k = 1, \ldots, q$ of covariate component X_k does not have a specific meaning.

- Nevertheless, adding positional encodings can be beneficial. Empirical evidence has shown that this facilitates learning the interactions between the covariate components.



4 The credibility Transformer

OVERVIEW

- The predictive performance of the Transformer on tabular data can further be improved by integrating a credibility mechanism.
- This credibility mechanism is inspired by *Bühlmann credibility*; Bühlmann (1967) and Bühlmann and Straub (1970). It balances the impact of instance-individual information and global portfolio behavior.
- This architecture is called *credibility Transformer* (CT), and it is one of the most powerful network architectures on tabular data nowadays.

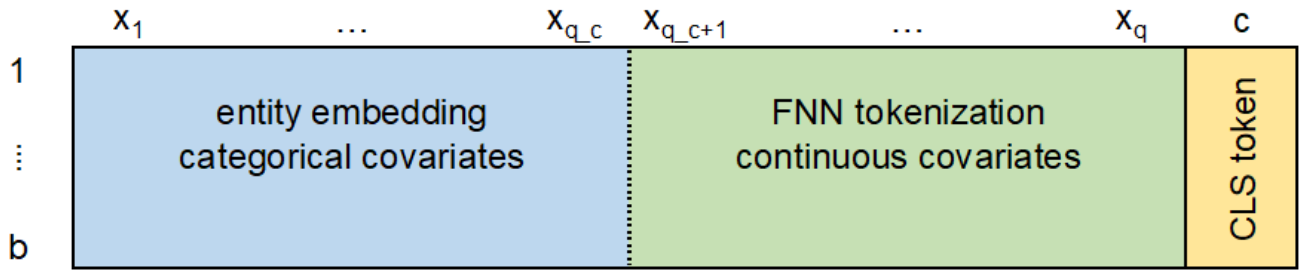
The credibility Transformer was introduced in Richman, Scognamiglio and Wüthrich (2025).

4.1 The augmented input tensor, revisited

- We start from the *augmented input tensor*

$$\boldsymbol{X}_{1:q+1} = \begin{bmatrix} \boldsymbol{X}^{\circ}_{1:q} \\ \boldsymbol{c}^{\top} \end{bmatrix} = \left[\boldsymbol{e}_1^{\text{EE}}, \ldots, \boldsymbol{e}_{q_c}^{\text{EE}}, \boldsymbol{z}_{q_c+1}, \ldots, \boldsymbol{z}_q, \boldsymbol{c} \right]^{\top} \in \mathbb{R}^{(q+1)\times b}.$$

- Recall: the CLS token $\boldsymbol{c}$ with components c_k are used to encode the columns k of the input tensor $\boldsymbol{X}^{\circ}_{1:q}$.



- $q+1$ reflects the time direction t, and there are b channels; the above shows the transposed 2D tensor $\boldsymbol{X}^{\top}_{1:q+1}$.

4.2 Transformer layer and CLS token encoding

- The augmented input tensor $\boldsymbol{X}_{1:q+1}$ is processed through the Transformer layer

$$\boldsymbol{z}^{\text{trans}} : \mathbb{R}^{(q+1)\times b} \to \mathbb{R}^{(q+1)\times b}, \qquad \boldsymbol{X}_{1:q+1} \mapsto \boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:q+1}),$$

  taking the structure as introduced above:

$$\boldsymbol{z}^{\text{trans}}(\boldsymbol{X}_{1:q+1}) = \boldsymbol{z}^{\text{skip}}(\boldsymbol{X}_{1:q+1}) + \left(\boldsymbol{z}^{\text{t-FNN}} \circ \boldsymbol{z}^{\text{norm}}\right)\left(\boldsymbol{z}^{\text{skip}}\left(\boldsymbol{X}_{1:q+1}\right)\right).$$

- Note, Richman, Scognamiglio and Wüthrich (2025) consider a more complicated architecture which, in essence, serves the same purpose.
- For further processing, we only read-out the transformed CLS token

$$\boldsymbol{c}^{\text{trans}}(\boldsymbol{X}) = \boldsymbol{z}^{\text{trans}}_{q+1}(\boldsymbol{X}_{1:q+1}) \in \mathbb{R}^b.$$

- This CLS token is *fully data driven*, as it encodes the tensor $\boldsymbol{X}_{1:q+1}$.

4.3 Prior CLS token

- The above CLS token $\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})$ is fully data driven.
- We define its prior counterpart (not considering the covariates $\boldsymbol{X}^{\circ}_{1:q}$).
- For this, we take the value $\boldsymbol{v}_{q+1}$ of the CLS token $\boldsymbol{c}$ *before interacting* with the covariates $\boldsymbol{X}^{\circ}_{1:q}$.
- This prior value of the CLS token is given by

$$\boldsymbol{v}_{q+1} = \boldsymbol{z}_V^{\text{FNN}}(\boldsymbol{X}_{q+1}) = \boldsymbol{z}_V^{\text{FNN}}(\boldsymbol{c}) \in \mathbb{R}^b.$$

  One could still add the skip connections, but it has turned out to not be necessary.
- The *prior token* is obtained by running the value $\boldsymbol{v}_{q+1}$ through the same chain of transformation (with identical parameters as the Transformer) to bring it into the *CLS token space*

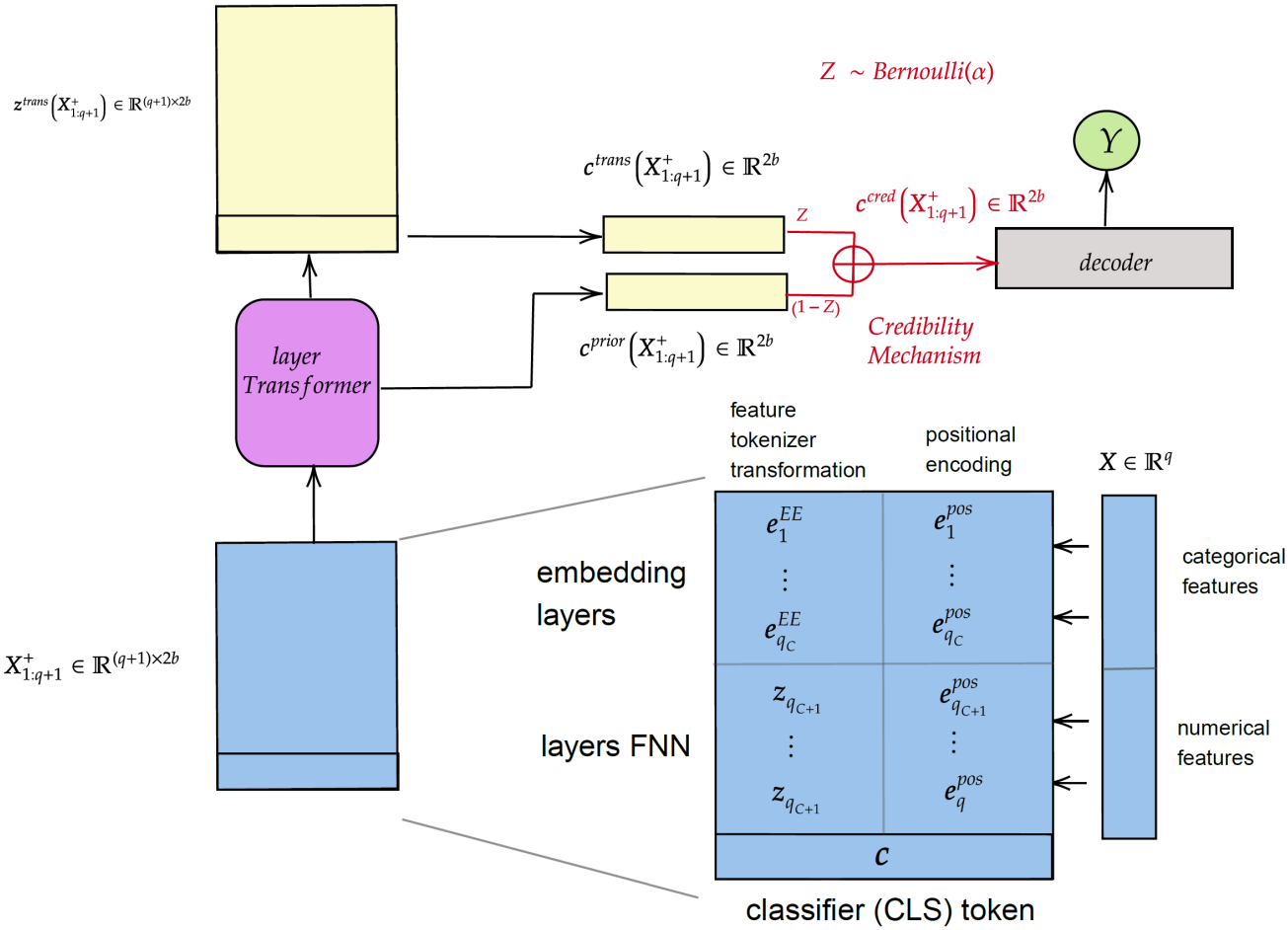
$$\boldsymbol{c}^{\text{prior}} = \big(\boldsymbol{z}^{\text{FNN}} \circ \boldsymbol{z}^{\text{norm}}\big)(\boldsymbol{v}_{q+1}) \in \mathbb{R}^b.$$

4.4 Credibility-inspired mechanism

- In the credibility Transformer, both tokens $\boldsymbol{c}^{\text{prior}}$ and $\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})$ are used for prediction.
- During SGD training, we randomly select between the *prior token* $\boldsymbol{c}^{\text{prior}}$ and the *data driven token* $\boldsymbol{c}^{\text{trans}}(\boldsymbol{X})$.
- Select a fixed probability weight $\alpha \in [0,1]$.
- During each SGD iteration, sample new i.i.d. Bernoulli random variables $Z \sim \text{Bernoulli}(\alpha)$.
- Set during SGD training

$$\boldsymbol{c}^{\text{cred}} = Z\,\boldsymbol{c}^{\text{trans}}(\boldsymbol{X}) + (1-Z)\,\boldsymbol{c}^{\text{prior}} \in \mathbb{R}^b.$$

- Thus, we randomly select the prior or the data driven token, allowing to train both of them simultaneously.



- The last step of the credibility Transformer is the readout (decoder).
- For this, we run the credibility token $\boldsymbol{c}^{\text{cred}}(\boldsymbol{X})$ through a FNN decoder. E.g., we may consider

$$\boldsymbol{X} \mapsto \mu^{\text{cred}}(\boldsymbol{X}) = g^{-1}\left\langle \boldsymbol{w}, \boldsymbol{z}^{\text{FNN}}(\boldsymbol{c}^{\text{cred}}(\boldsymbol{X}))\right\rangle.$$

- For the implementation of all these modules we refer to our website Wüthrich *et al.* (2025), where a full implementation of the credibility Transformer is available.

4.5 Remarks on the credibility Transformer

- The random weighting scheme is only applied during SGD training; this acts like drop-out. It helps to stabilize the training procedure.

- For each iteration of the SGD algorithm, the i.i.d. Bernoulli random variables are re-sampled.
- For prediction, set $Z = 1$ and $\alpha = 1$, respectively.
- One can verify by setting $Z = 0$ in the prediction, that the prior token $\boldsymbol{c}^{\text{prior}}$ learns the null model not including any covariates. Thus, it learns the *global mean*.
- As a result, we receive a weighted average between a prior model (null model) and a data driven model (covariates regression model).
- Probability α is a hyper-parameter that needs to be selected by the modeler (by cross-validation). In our case, $\alpha = 0.95$ provided good results.

4.6 Example: Credibility Transformer

model	***in-sample***	***out-of-sample***	***balance (in %)***
null model	47.722	47.967	7.36
GLM	45.585	45.435	7.36
FNN (embedding)	45.030	44.948	7.16
ensembling FNN	44.636	**44.864**	7.36
credibility Transformer	44.710	44.716	7.45
ensembling CT	44.577	**44.635**	7.24

- For the ensembling predictor we use 10 individual network fits.
- Out-of-sample standard deviation of the credibility Transformer is 0.071. Thus, improvement is significant.
- Even a single run of the credibility Transformer outperforms the ensembling predictor of standard FNNs.

4.7 The hidden credibility mechanism

- The credibility transformer introduces a second less obvious credibility mechanism which is realized during the dot-product attention.
- To understand its functioning, consider the last row vector of the attention head

$$H_{q+1} = A_{q+1}V \in \mathbb{R}^b.$$

- I.e., the last row of the attention head, H_{q+1}, is obtained by multiplying the row vector $A_{q+1} = (a_{q+1,1}, \ldots, a_{q+1,q+1})$ with the corresponding columns of the value matrix V.
- Furthermore, recall the softmax normalization entering A_{q+1}

$$\sum_{j=1}^{q+1} a_{q+1,j} = 1, \qquad \text{with weights} \quad a_{q+1,j} > 0.$$

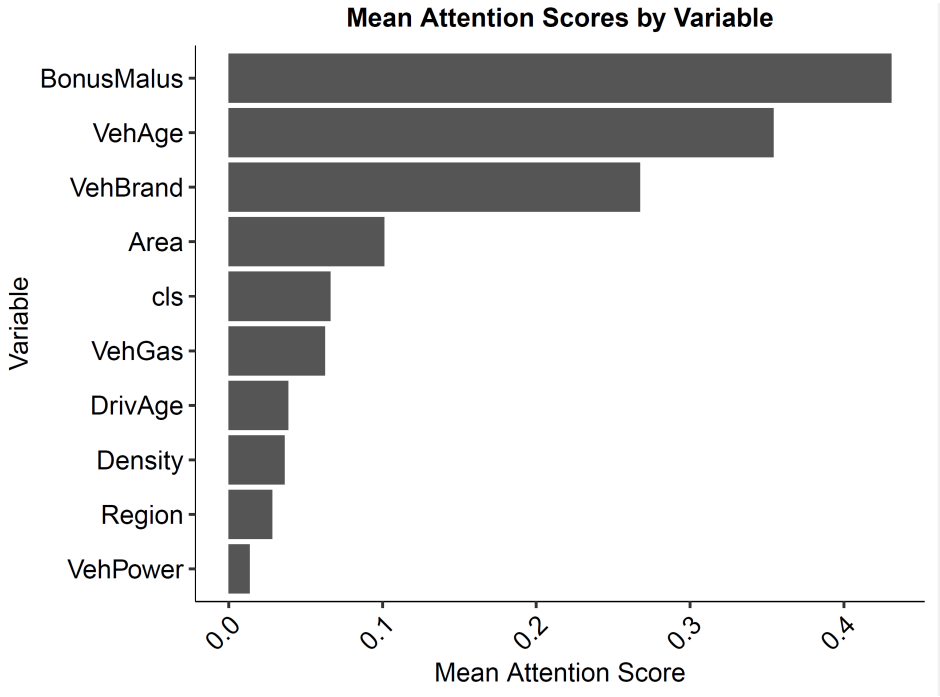
- This allows us to compute the row vector

$$\begin{aligned} H_{q+1} \;=\; A_{q+1}V &= \left(\sum_{j=1}^{q} a_{q+1,j} V_{j,k} + a_{q+1,q+1} V_{q+1,k} \right)_{k=1}^{b} \\ &= \left(P \sum_{j=1}^{q} \frac{a_{q+1,j}}{P}\, V_{j,k} + (1-P)\, V_{q+1,k} \right)_{k=1}^{b} \\ &= P\, \boldsymbol{v}^{\text{covariate}} + (1-P)\, \boldsymbol{v}_{q+1}^{\top}, \end{aligned}$$

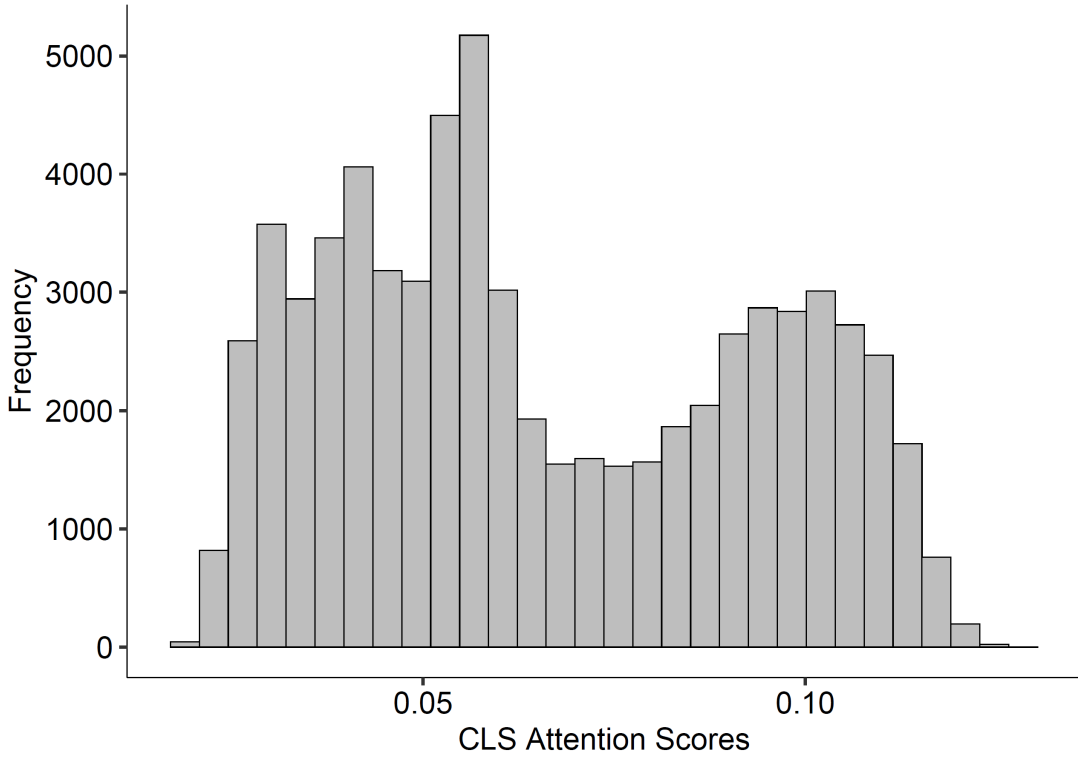
  for

  - *credibility weight* $1 - P := a_{q+1,q+1} \in (0,1)$,
  - *observation driven estimate* $\boldsymbol{v}^{\text{covariate}}$, and
  - *prior estimate* $\boldsymbol{v}_{q+1}$.
- This formulation reveals that the attention mechanism for the CLS token can be interpreted as a *credibility-weighted average* of the covariate-driven information $\boldsymbol{v}^{\text{covariate}}$ and a prior value $\boldsymbol{v}_{q+1}$.

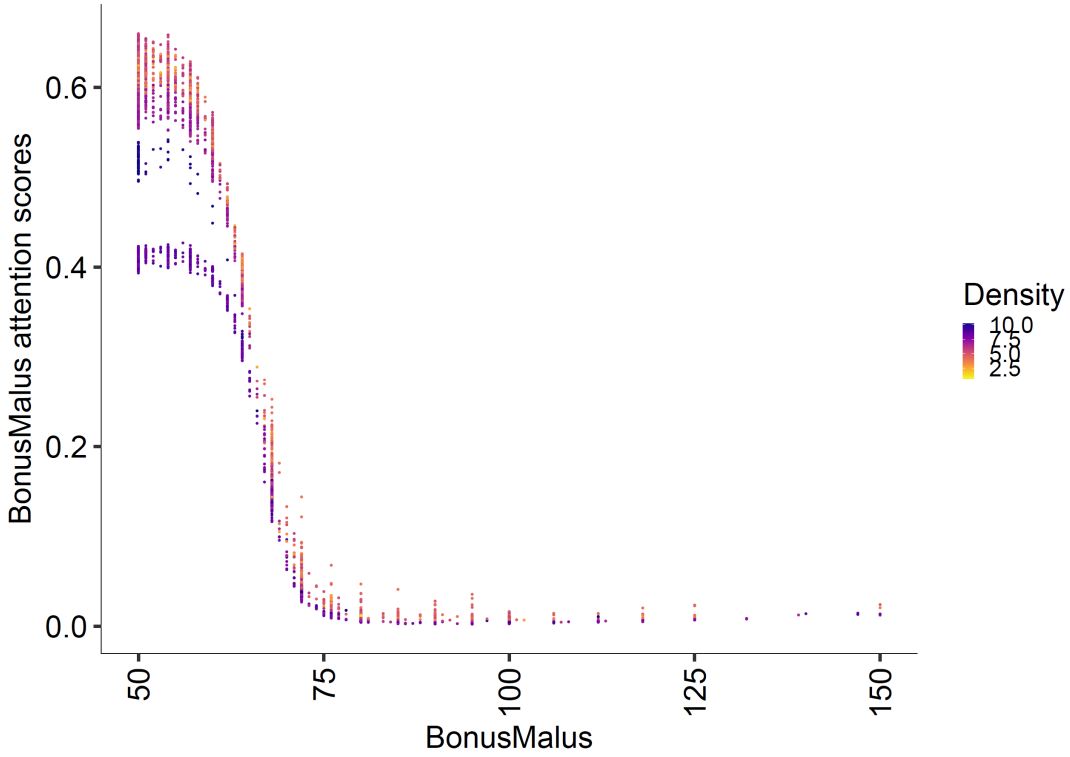
- This consideration also allows for a variable importance measure.
- In our examples we allocated roughly a credibility weight of $1 - P = 7\%$ to the prior and $P = 93\%$ to the observation driven part.



- Distribution of CLS token attention scores across all instances.



- Attention score for `BonusMalus` colored by `log-Density`.



4.8 Conclusions and summary

- We introduced the Transformer architecture …
- … which we enhanced with dual credibility mechanisms:
  - Explicit mechanism through probabilistic CLS token selection during training (α parameter)
  - Implicit mechanism through attention weights functioning as learned credibility formula
- We demonstrated integration of modern deep learning components:
  - Attention mechanism
  - Transformer layers
  - Tokenization and embedding
  - Positional encoding

- What lessons can we draw from this work?
  - Even in a state-of-the-art deep learning model, applying classical actuarial principles can lead to gains in model performance.
  - Actuarial science and modern machine learning aren't separate disciplines…
  - … but rather, classical actuarial principles like credibility still have deep relevance in the age of deep learning.

5 Appendix: Drop-out layer

OVERVIEW

- *Drop-out* is a widely used regularization technique in network fitting.
- We have already mentioned it a couple of times.
- It randomly removes neurons (units) during model training to enhance the model's generalization capabilities.

Drop-out has been introduced by Srivastava *et al.* (2014) and Wager, Wang and Liang (2013).

- Drop-out is typically implemented by multiplying the output of a specific layer by i.i.d. realizations of Bernoulli random variables with a fixed drop-out rate $\alpha \in (0, 1)$ in each step of SGD training.
- Drop-out is formalized, e.g., in the case of a FNN/tabular output layer by

$$\boldsymbol{z}^{\text{drop}} : \mathbb{R}^q \to \mathbb{R}^q, \qquad \boldsymbol{X} \mapsto \boldsymbol{z}^{\text{drop}}(\boldsymbol{X}) = \boldsymbol{Z} \odot \boldsymbol{X},$$

  where $\boldsymbol{Z} = (Z_1, Z_2, \ldots, Z_q)^\top \in \{0, 1\}^q$ is a vector of i.i.d. Bernoulli random

variables that are re-sampled in each SGD step, and $\odot$ denotes the element-wise Hadamard product.

Copyright

- © The Authors
- This notebook and these slides are part of the project "AI Tools for Actuaries". The lecture notes can be downloaded from:

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5162304

- This material is provided to reusers to distribute, remix, adapt, and build upon the material in any medium or format for noncommercial purposes only, and only so long as attribution and credit is given to the original authors and source, and if you indicate if changes were made. This aligns with the Creative Commons Attribution 4.0 International License CC BY-NC.

References

Ba, J.L., Kiros, J.R. and Hinton, G.E. (2016) "Layer normalization," *arXiv:1607.06450* [Preprint]. Available at: https://arxiv.org/abs/1607.06450.

Bühlmann, H. (1967) "Experience rating and credibility," *ASTIN Bulletin - The Journal of the IAA*, 4(3), pp. 199–207. Available at: https://www.cambridge.org/core/product/B9A879CE4A73397C653963B474A7F954.

Bühlmann, H. and Straub, E. (1970) "Glaubwürdigkeit für Schadensätze," *Mitteilungen der Schweizerischen Vereinigung der Versicherungsmathematiker*, 1970, pp. 111–131. Available at: https://www.e-

periodica.ch/digbib/view?pid=msa-001%3A1970%3A70%3A%3A115.

Devlin, J. *et al.* (2019) "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Available at: https://aclanthology.org/N19-1423/.

Gorishniy, Y., Kotelnikov, A. and Babenko, A. (2024) "TabM: Advancing tabular deep learning with parameter-efficient ensembling," *arXiv:2410.24210* [Preprint]. Available at: https://arxiv.org/abs/2410.24210.

Gorishniy, Y., Rubachev, I. and Babenko, A. (2022) "On embeddings for numerical features in tabular deep learning," *arXiv preprint arXiv:2203.05556* [Preprint]. Available at: https://arxiv.org/abs/2203.05556.

Ioffe, S. and Szegedy, C. (2015) "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning*. PMLR, pp. 448–456. Available at: https://proceedings.mlr.press/v37/ioffe15.html.

Richman, R., Scognamiglio, S. and Wüthrich, M.V. (2025) "The credibility transformer," *European Actuarial Journal*, pp. 1–35. Available at: https://link.springer.com/article/10.1007/s13385-025-00413-y.

Srivastava, N. *et al.* (2014) "Dropout: A simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, 15(1), pp. 1929–1958. Available at: https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf.

Vaswani, A. *et al.* (2017) "Attention is all you need," *Advances In Neural Information Processing Systems*, 30. Available at: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Wager, S., Wang, S. and Liang, P.S. (2013) "Dropout training as adaptive regularization," *Advances in Neural Information Processing Systems*, 26. Available at: https://arxiv.org/abs/1307.1493.

Wüthrich, M.V. *et al.* (2025) "AI Tools for Actuaries," *SSRN Manuscript* [Preprint]. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5162304.